

12.1: Graph Data Management



UMD DATA605 - Big Data Systems

12.1: Graph Data Management

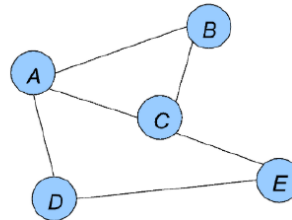
– **Instructor:** Dr. GP Saggese, gsaggese@umd.edu

1 / 15

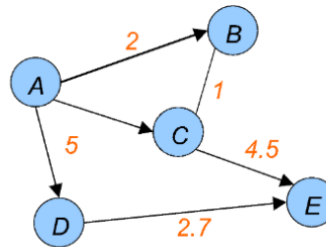
2 / 15: Graphs: Background

Graphs: Background

- A **graph** represents entities and their connections
 - *Entities* are vertices (nodes)
 - *Connections* are edges (links, arcs, relationships)
- **Applications of graphs in many fields**
 - Social networks
 - Biological networks
 - Information networks
 - Infrastructure networks
 - ...



An undirected, unweighted graph



A directed, edge-weighted graph

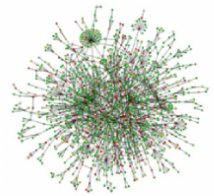
2 / 15

- **Graphs: Background**
 - A *graph* is a mathematical structure used to model pairwise relations between objects.
 - **Entities** are represented as vertices (or nodes). These are the individual objects or points in the graph.
 - **Connections** are represented as edges (or links, arcs, relationships). These show how the entities are related or connected to each other.
- **Applications of graphs in many fields**
 - Graphs are versatile and used in various domains:
 - **Social networks:** Representing connections between people or organizations.
 - **Biological networks:** Modeling interactions between biological entities like proteins or genes.
 - **Information networks:** Structuring data such as the World Wide Web or citation networks.
 - **Infrastructure networks:** Mapping out systems like transportation or utility networks.
- **Images Explanation**
 - The first image shows an *undirected, unweighted graph*, where edges have no direction and all connections are equal.
 - The second image illustrates a *directed, edge-weighted graph*, where edges have a direction and weights indicate the strength or capacity of the connection.

3 / 15: Graph Data Structures: Motivation

Graph Data Structures: Motivation

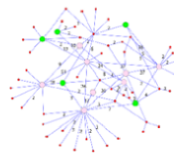
- **Graph data:**
 - Increasing volumes of data
 - Increasing interest in querying and reasoning
- **Sectors**
 - Healthcare
 - Finance
 - Logistics
- **Example applications**
 - Fraud detection
 - Recommendation systems
 - Network analysis



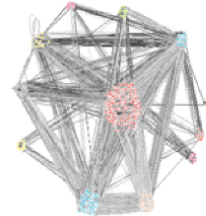
Protein-protein interaction network



Stock trading network



Supreme court citation network



Social networks

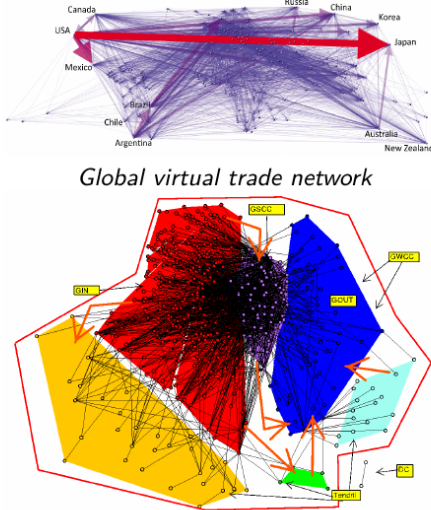
3 / 15

- **Graph data:**
 - With the explosion of data in recent years, there's a growing need to manage and analyze this information effectively. Graph data structures are particularly useful because they can represent complex relationships and interactions within data.
 - There's an increasing interest in using graph data for querying and reasoning, which means extracting meaningful insights and making informed decisions based on the data's structure.
- **Sectors:**
 - Graph data structures are being utilized across various sectors due to their ability to model relationships and interactions.
 - *Healthcare*: Used for understanding complex biological networks, such as protein interactions.
 - *Finance*: Helps in analyzing trading networks and detecting fraudulent activities.
 - *Logistics*: Optimizes supply chain networks and improves efficiency.
- **Example applications:**
 - *Fraud detection*: Graphs can reveal unusual patterns and connections that might indicate fraudulent behavior.
 - *Recommendation systems*: By analyzing user interactions and preferences, graphs can suggest relevant products or content.
 - *Network analysis*: Used to study and optimize various types of networks, from social media to transportation systems.

The images illustrate different types of networks, showcasing the versatility of graph data structures in representing complex systems.

Graph Data Structures: Motivation

- **Traditional tools** (e.g., relational DBs, NoSQL DBs) struggle with:
 - Storing and querying graph data
 - Processing graph-structured queries
- **Dedicated solutions** to:
 - Storing: Neo4j
 - Processing: Google Pregel, Apache Giraph, Spark GraphX



Global virtual trade network

Federal funds networks

4 / 15

-

- 4 / 15

5 / 15: Knowledge Graphs

Knowledge Graphs

- **Representation of knowledge in the form of graphs**
 - Capture entities, relationships, properties
 - Provide structured view of real-world information
 - E.g., Google Knowledge Graph, DBpedia, Wikidata
 - E.g., RDF or Property Graph models
- **Applications**
 - Enable machine understanding of complex domains
 - Support semantic search, recommendation, analytics
 - Used in industries for data integration, knowledge discovery, AI applications
- **Ontologies**
 - Provide formal and structure representation of knowledge
 - E.g., type of things and how they relate to each other
 - Like “meaning and rules” to interpret data into a knowledge graph
 - Promote interoperability across knowledge bases

5 / 15

- **Knowledge Graphs**
 - *Representation of knowledge in the form of graphs:* Knowledge graphs are a way to organize information using nodes and edges, where nodes represent entities (like people, places, or things) and edges represent the relationships between them. This structure helps in capturing complex relationships and properties of entities, offering a more intuitive and interconnected view of data.
 - They provide a structured view of real-world information, making it easier to understand and analyze. For example, the Google Knowledge Graph helps improve search results by connecting related information. Other examples include DBpedia and Wikidata, which are large-scale knowledge bases.
 - Knowledge graphs can be built using models like RDF (Resource Description Framework) or Property Graphs, which define how data is stored and accessed.
- **Applications**
 - Knowledge graphs enable machines to understand complex domains by providing context and relationships between data points. This understanding is crucial for tasks like semantic search, where the goal is to find information based on meaning rather than just keywords.
 - They support recommendation systems by identifying related items or concepts, and they enhance analytics by uncovering hidden patterns and insights.
 - Industries use knowledge graphs for data integration, allowing different data sources to work together seamlessly, and for knowledge discovery, which involves finding new insights from existing data. They are also integral to AI applications that require a deep understanding of data.
- **Ontologies**

- Ontologies provide a formal and structured representation of knowledge, defining the types of things and how they relate to each other. They act like a set of rules and meanings that help interpret data into a knowledge graph.
- By defining these relationships and meanings, ontologies promote interoperability across different knowledge bases, ensuring that data from various sources can be combined and understood in a consistent manner. This is essential for creating comprehensive and accurate knowledge graphs.

6 / 15: Graph Data Models: RDF

Graph Data Models: RDF

- **Resource Description Framework**

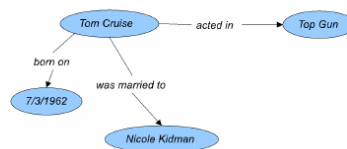
- RDF uses triples:
subject-predicate-object
- Connects “subject” and “object”
through “predicate”
- E.g., “TomCruise-acted-TopGun”

- **Used to represent knowledge bases**

- Queried through SPARQL

- **Pros**

- Standardization
 - W3C standard to model data
 - Subject and object can be URIs in semantic web
- Interoperability
 - Merge RDF data stores
- Extensibility
 - Add new nodes and relationships
 - Support ontologies



6 / 15

- **Resource Description Framework (RDF)**

- RDF is a framework used to represent information about resources in the web.
- It uses a structure called *triples*, which consist of three parts: subject, predicate, and object.
- These triples connect a “subject” to an “object” through a “predicate,” forming a simple statement. For example, “TomCruise-acted-TopGun” indicates that Tom Cruise acted in the movie Top Gun.

- **Used to Represent Knowledge Bases**

- RDF is commonly used to create knowledge bases that can be queried using a language called SPARQL. This allows users to retrieve and manipulate data stored in RDF format efficiently.

- **Pros**

- **Standardization**

- RDF is standardized by the World Wide Web Consortium (W3C), ensuring a consistent way to model data.
- Both subjects and objects can be Uniform Resource Identifiers (URIs), which are

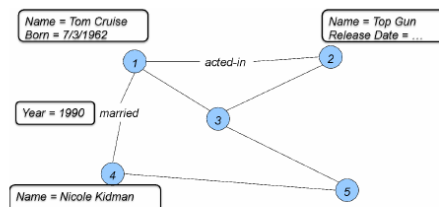
essential for the semantic web.

- **Interoperability**
 - RDF allows for the merging of different RDF data stores, facilitating data integration across various sources.
- **Extensibility**
 - It is easy to add new nodes and relationships in RDF, making it flexible.
 - RDF supports ontologies, which help define the relationships between different data elements, enhancing data understanding and reuse.

7 / 15: Graph Data Models: Property Graph

Graph Data Models: Property Graph

- Directed graph with nodes and edges having key-values *properties*
 - Similar expressive power to RDFs
- **No universal standard**
 - Less “schema”
 - Harder to interoperate
- **Examples of query languages**
 - Cypher for Neo4j
 - Gremlin for Apache TinkerPop



7 / 15

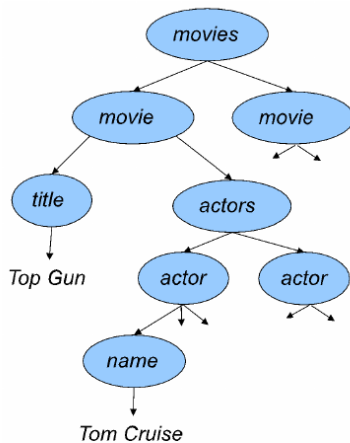
- **Graph Data Models: Property Graph**
 - A property graph is a type of directed graph where both nodes and edges can have associated key-value pairs known as *properties*. This allows for rich data representation, similar in expressive power to RDFs (Resource Description Frameworks).
- **No Universal Standard**
 - Property graphs lack a universal standard, which means there is less emphasis on a fixed *schema*. This flexibility can be advantageous for certain applications but makes it harder to ensure interoperability between different systems or databases.
- **Examples of Query Languages**
 - *Cypher* is a popular query language used with Neo4j, a leading graph database that leverages property graphs.
 - *Gremlin* is another query language, used with Apache TinkerPop, which is a graph computing framework that supports property graphs.
- **Image Explanation**
 - The image illustrates a simple property graph. Nodes represent entities like people or

movies, and edges represent relationships such as “acted-in” or “married.” Each node and edge can have properties, such as names, dates, or years, providing detailed context for the relationships.

8 / 15: Graph Data Models: XML

Graph Data Models: XML

- Common data model for flexible data representation
 - Directed labeled tree
 - Popular for non-tabular data exchange



```

<movies>
  <movie>
    <title>Top Gun</title>
    <actors>
      <actor>
        <name>Tom Cruise</name>
        <born>7/3/1962</born>
      </actor>
      <actor>
        ...
      </actor>
    </actors>
  </movie>

```

8 / 15

- **Graph Data Models: XML**
 - XML is a *common data model* used for flexible data representation. It is particularly useful for scenarios where data does not fit neatly into tables, such as hierarchical or nested data structures.
 - XML structures data as a **directed labeled tree**, where each node represents a data element, and edges represent relationships between these elements. This makes it ideal for representing complex data relationships.
 - XML is *popular for non-tabular data exchange* because it allows for a clear and structured way to represent data that can be easily shared and understood across different systems and platforms.
- **Example XML Structure**
 - The provided XML snippet illustrates a simple structure for representing movies and their actors.
 - The root element `<movies>` contains multiple `<movie>` elements, each with its own `<title>` and `<actors>`.
 - Each `<actor>` element includes details like `<name>` and `<born>`, showcasing how XML can encapsulate detailed information in a hierarchical manner.
 - The accompanying diagram visually represents this XML structure as a tree, highlighting the parent-child relationships between elements.

9 / 15: Comparison

Comparison

Feature	RDF	Property Graph	XML
Core data model	Triples: (subject, predicate, object)	Nodes and edges with properties	Hierarchical tree of elements
How facts are stored	Fact is a separate triple	Properties on nodes/edges	Nested tags with attributes
Attributes	Modeled as triples	Key-value pairs	Attributes or child elements
Semantics	Formal (RDF, RDFS, OWL)	No built-in semantics	No built-in semantics
Ontology support	Native, standardized	Optional	None (only schema)
Reasoning & inference	Built-in (e.g., OWL)	Usually not supported	Not supported
Data integration	Excellent for heterogeneous data	Manual mapping	Hard across schemas
Query language	SPARQL	Cypher, Gremlin, GQL	XPath, XQuery
Query style	Pattern matching	Traversals	Tree navigation
Schema	Optional, supported	Labels, constraints	XSD, DTD
Standards	W3C standards	Vendor-specific	W3C standards
Interoperability	Very high	Limited	High for documents
Traversal performance	Moderate	Very fast	Poor
Use cases	Linked data, KBs	Fraud, recommendations	Docs, configs, data exchange
Examples	Wikidata, DBpedia	Neo4j, Neptune	SOAP, RSS, Office

9 / 15

- **Core data model:**
 - *RDF* uses a simple structure called triples, which consist of a subject, predicate, and object. This is like a basic sentence structure.
 - *Property Graph* uses nodes and edges, where nodes represent entities and edges represent relationships, both of which can have properties.
 - *XML* organizes data in a hierarchical tree, similar to a family tree, with elements nested within each other.
- **How facts are stored:**
 - In *RDF*, each fact is stored as a separate triple, making it easy to add or remove facts.
 - *Property Graph* stores facts as properties on nodes or edges, allowing for rich, detailed descriptions.
 - *XML* uses nested tags with attributes to store facts, which can be complex but is very structured.
- **Attributes:**
 - In *RDF*, attributes are also modeled as triples, maintaining a consistent structure.
 - *Property Graph* uses key-value pairs for attributes, which is straightforward and flexible.
 - *XML* can use either attributes or child elements to represent additional information.
- **Semantics:**
 - *RDF* has formal semantics, meaning it follows strict rules and standards (like RDF, RDFS, OWL) for meaning.
 - *Property Graph* and *XML* do not have built-in semantics, meaning their meaning is not standardized.
- **Ontology support:**
 - *RDF* supports ontologies natively and is standardized, which helps in defining complex

- relationships.
- *Property Graph* offers optional ontology support, depending on the implementation.
- *XML* does not support ontologies, only schemas.
- **Reasoning & inference:**
 - *RDF* has built-in reasoning and inference capabilities, allowing it to derive new information from existing data.
 - *Property Graph* and *XML* usually do not support reasoning or inference.
- **Data integration:**
 - *RDF* excels at integrating heterogeneous data, making it ideal for combining different data sources.
 - *Property Graph* requires manual mapping for integration.
 - *XML* finds it challenging to integrate data across different schemas.
- **Query language:**
 - *RDF* uses SPARQL, a powerful language for querying triples.
 - *Property Graph* uses languages like Cypher, Gremlin, or GQL for querying.
 - *XML* uses XPath and XQuery for navigating and querying data.
- **Query style:**
 - *RDF* uses pattern matching to find data.
 - *Property Graph* uses traversals, moving through nodes and edges.
 - *XML* uses tree navigation to access data.
- **Schema:**
 - *RDF* has optional schema support.
 - *Property Graph* uses labels and constraints for schema.
 - *XML* uses XSD or DTD for defining schemas.
- **Standards:**
 - *RDF* and *XML* follow W3C standards, ensuring wide acceptance and compatibility.
 - *Property Graph* standards are often vendor-specific, leading to variations.
- **Interoperability:**
 - *RDF* offers very high interoperability, making it easy to work with other systems.
 - *Property Graph* has limited interoperability due to vendor-specific implementations.
 - *XML* has high interoperability for document exchange.
- **Traversal performance:**
 - *RDF* has moderate performance for data traversal.
 - *Property Graph* is very fast at traversing data.
 - *XML* performs poorly in traversal due to its hierarchical nature.
- **Use cases:**
 - *RDF* is used for linked data and knowledge bases.
 - *Property Graph* is ideal for fraud detection and recommendation systems.
 - *XML* is used for documents, configurations, and data exchange.
- **Examples:**
 - *RDF* examples include Wikidata and DBpedia.
 - *Property Graph* examples include Neo4j and Amazon Neptune.
 - *XML* examples include SOAP, RSS, and Microsoft Office documents.

10 / 15: Storing Graph Data

Storing Graph Data

- **File systems**
 - Simple
 - No transactions, ACID compliance
 - Minimal functionality (build analysis/querying on top)
- **Relational databases**
 - Mature technology
 - SQL, transactions, ACID compliance, toolchains
 - Minimal functionality for graph data
- **NoSQL key-value stores**
 - Handle large datasets efficiently in a distributed fashion
 - Minimal native functionality for graph data
- **Graph databases**
 - Efficiently support complex queries/tasks (e.g., graph traversals)
 - Less mature than RDBMSs
 - Often lack declarative language like SQL
 - Write custom programs

10 / 15

- **Storing Graph Data**
- **File systems**
 - *Simple*: File systems are straightforward to use for storing data, as they allow you to save files directly on your computer or server.
 - *No transactions, ACID compliance*: They do not support transactions or ACID (Atomicity, Consistency, Isolation, Durability) properties, which are important for ensuring data integrity and reliability.
 - *Minimal functionality*: File systems provide basic storage capabilities, but you need to build additional tools for data analysis or querying.
- **Relational databases**
 - *Mature technology*: Relational databases have been around for decades and are well-established in the industry.
 - *SQL, transactions, ACID compliance, toolchains*: They support SQL for querying, transactions for data integrity, and ACID compliance, making them reliable for many applications.
 - *Minimal functionality for graph data*: While powerful, they are not specifically designed for graph data, which can limit their effectiveness for such tasks.
- **NoSQL key-value stores**
 - *Handle large datasets efficiently in a distributed fashion*: These databases are designed to manage large volumes of data across multiple servers, making them scalable.
 - *Minimal native functionality for graph data*: They are not inherently equipped to handle

graph-specific operations, requiring additional work to manage graph data.

- **Graph databases**

- *Efficiently support complex queries/tasks (e.g., graph traversals)*: Specifically designed for graph data, these databases excel at handling complex queries like traversals.
- *Less mature than RDBMSs*: They are newer compared to relational databases and may not have the same level of maturity or widespread adoption.
- *Often lack declarative language like SQL*: Many graph databases do not have a standard query language like SQL, which means you might need to write custom programs to interact with the data.

11 / 15: Graph Databases

Graph Databases

- Many specialized graph DB systems
 - E.g., Neo4j, Titan, OrientDB, AllegroGraph
- Key distinctions from relational / NoSQL databases
 - Store graph structure with pointers
 - Avoid joins
 - Simplify graph traversals
 - Manage and query graph-structured data
 - Write queries and graph algorithms (reachability, shortest paths)
 - Support graph query languages: SPARQL, Cypher, Gremlin
 - Declarative interfaces
 - Provide programmatic API for arbitrary graph algorithms

11 / 15

- **Graph Databases**

- Graph databases are a type of database specifically designed to handle data that is interconnected, much like a network or web. They are particularly useful for applications where relationships between data points are as important as the data itself.
- **Many specialized graph DB systems**
 - There are several specialized systems for managing graph databases. **Neo4j**, **Titan**, **OrientDB**, and **AllegroGraph** are examples of popular graph database systems. Each of these systems has its own strengths and is chosen based on specific needs and use cases.
- **Key distinctions from relational / NoSQL databases**
 - **Store graph structure with pointers**
 - Unlike traditional relational databases that use tables and rows, graph databases store data in nodes and edges, using pointers to directly connect related data. This structure helps in avoiding complex joins, which can be computationally expensive.

- This design simplifies *graph traversals*, making it easier and faster to navigate through connected data.
- **Manage and query graph-structured data**
 - Graph databases are optimized for managing and querying data that is inherently structured as a graph. This includes writing queries and performing graph algorithms like *reachability* (finding if a path exists between nodes) and *shortest paths* (finding the shortest route between nodes).
- **Support graph query languages: SPARQL, Cypher, Gremlin**
 - These databases support specialized query languages designed for graph data. **SPARQL**, **Cypher**, and **Gremlin** are examples of such languages, each offering unique features for querying and manipulating graph data.
- **Declarative interfaces**
 - Graph databases often provide declarative interfaces, allowing users to specify *what* they want to achieve without detailing *how* to accomplish it. This abstraction simplifies complex queries and operations.
- **Provide programmatic API for arbitrary graph algorithms**
 - They offer programmatic APIs that allow developers to implement custom graph algorithms, providing flexibility to perform complex operations tailored to specific application needs. This is crucial for applications that require advanced data analysis and manipulation.

12 / 15: Query Languages for Graph Databases

Query Languages for Graph Databases

Feature	Cypher	Gremlin	SPARQL
Data Model	Property Graph	Property Graph	RDF Triple
Query Style	Declarative	Imperative	Declarative
Syntax Example	SQL-like	Functional API	Triple patterns
Best For	Pattern matching	Complex traversals	Semantic data
Standardization	Neo4j-specific (OpenCypher)	Apache TinkerPop	W3C Standard
Backend Support	Mainly Neo4j	Multi-platform (TinkerPop)	RDF stores
Learning Curve	Low	High	Medium
Use Cases	Social graphs, fraud detection	Distributed graph processing	Linked data, knowledge graphs

12 / 15

- **Query Languages for Graph Databases:** This slide compares three popular query languages used in graph databases: Cypher, Gremlin, and SPARQL. Each language has unique features and is suited for different types of graph data and use cases.

- **Data Model:**
 - **Cypher** and **Gremlin** both use the *Property Graph* model, which represents data as nodes and edges with properties.
 - **SPARQL** uses the *RDF Triple* model, which represents data as subject-predicate-object triples, ideal for semantic data.
- **Query Style:**
 - **Cypher** and **SPARQL** are *declarative*, meaning you specify what you want, not how to get it.
 - **Gremlin** is *imperative*, meaning you describe the steps to retrieve the data.
- **Syntax Example:**
 - **Cypher** has an *SQL-like* syntax, making it familiar to those with SQL experience.
 - **Gremlin** uses a *Functional API*, which can be more complex.
 - **SPARQL** uses *Triple patterns*, aligning with its RDF data model.
- **Best For:**
 - **Cypher** excels in *pattern matching*, useful for finding relationships in data.
 - **Gremlin** is suited for *complex traversals*, handling intricate graph paths.
 - **SPARQL** is ideal for *semantic data*, such as linked data and knowledge graphs.
- **Standardization:**
 - **Cypher** is *Neo4j-specific* but has an open version called OpenCypher.
 - **Gremlin** is part of the *Apache TinkerPop* framework, supporting multiple platforms.
 - **SPARQL** is a *W3C Standard*, ensuring broad compatibility and support.
- **Backend Support:**
 - **Cypher** is mainly supported by *Neo4j*.
 - **Gremlin** is supported by *multi-platforms* through TinkerPop.
 - **SPARQL** is used in *RDF stores*, which are databases designed for RDF data.
- **Learning Curve:**
 - **Cypher** has a *low* learning curve, making it accessible for beginners.
 - **Gremlin** has a *high* learning curve due to its imperative nature.
 - **SPARQL** has a *medium* learning curve, balancing complexity and ease of use.
- **Use Cases:**
 - **Cypher** is great for *social graphs* and *fraud detection*.
 - **Gremlin** is used for *distributed graph processing*, handling large-scale data.
 - **SPARQL** is best for *linked data* and *knowledge graphs*, leveraging its semantic capabilities.

13 / 15: Query Languages: Cypher

Query Languages: Cypher

- **Purpose-Built for Property Graphs**
 - Nodes and relationships with key-value properties
- **Declarative Syntax**
 - Express *what* to retrieve, not *how*
- **Optimized for Pattern Matching**
 - Easily finds subgraph structures (e.g., friends of friends)
 - Limited in Graph Analytics
 - Not ideal for reachability, shortest paths, or centrality
- **Neo4j Native**
- Example:
 - Find the names of people who know someone named “Alice”

```

MATCH (person)-[:KNOWS]->(friend {name: "Alice"})
RETURN person.name

```

13 / 15

- **Query Languages: Cypher**
 - **Purpose-Built for Property Graphs**
 - Cypher is specifically designed to work with *property graphs*, which are a type of graph database. In these graphs, data is stored in nodes and relationships, each of which can have properties in the form of key-value pairs. This structure allows for a flexible and intuitive way to represent complex data relationships.
 - **Declarative Syntax**
 - Cypher uses a *declarative* approach, meaning you specify *what* data you want to retrieve rather than detailing *how* to get it. This makes it easier to write and understand queries, as you focus on the desired outcome rather than the process to achieve it.
 - **Optimized for Pattern Matching**
 - Cypher excels at finding specific patterns within a graph, such as identifying subgraph structures like “friends of friends.” However, it has limitations when it comes to more complex graph analytics tasks, such as calculating reachability, finding the shortest paths, or determining centrality measures. These tasks often require more specialized algorithms or tools.
 - **Neo4j Native**
 - Cypher is the native query language for Neo4j, a popular graph database platform. This means it is fully integrated and optimized for use with Neo4j, providing efficient and powerful querying capabilities for users of this system.
 - **Example:**
 - The example query demonstrates how to use Cypher to find the names of people who know someone named “Alice.” The **MATCH** clause specifies the pattern to look for: a

`person` node connected by a `KNOWS` relationship to a `friend` node with the property `name: "Alice"`. The `RETURN` clause then specifies that the query should output the `name` property of the `person` node. This example highlights Cypher's ability to express complex queries in a straightforward and readable manner.

14 / 15: Query Languages: Gremlin

Query Languages: Gremlin

- **Supports Multiple Models**
 - Compatible with both Property Graphs and RDF
- **Imperative Style**
 - Describes *how* to traverse the graph step by step
- **Traversal-Based Semantics**
 - Expresses computation as a flow of operations across vertices and edges
- Example:
 - *Find the names of people who know someone named "Alice"*

```
g.V().hasLabel('Person')
  .where(out('KNOWS').has('name', 'Alice'))
  .values('name')
```

14 / 15

- **Query Languages: Gremlin**
 - Gremlin is a query language designed for graph databases. It allows users to interact with and query graph data structures efficiently.
- **Supports Multiple Models**
 - *Compatible with both Property Graphs and RDF*: Gremlin is versatile because it can work with different types of graph models. Property Graphs are a common way to represent graph data with nodes and edges having properties. RDF (Resource Description Framework) is another model used mainly for semantic web applications. Gremlin's compatibility with both means it can be used in a wide range of applications and systems.
- **Imperative Style**
 - *Describes how to traverse the graph step by step*: Unlike declarative query languages like SQL, which focus on what data to retrieve, Gremlin is imperative. This means you specify the exact steps to navigate through the graph, giving you more control over the query process.
- **Traversal-Based Semantics**
 - *Expresses computation as a flow of operations across vertices and edges*: In Gremlin, queries are expressed as traversals. This means you define a sequence of operations that move through the graph's nodes (vertices) and connections (edges) to compute the desired result.

- **Example**

- *Find the names of people who know someone named “Alice”*: This example demonstrates a typical Gremlin query. It starts by selecting vertices labeled ‘Person’, then filters those who have an outgoing ‘KNOWS’ edge to a person named “Alice”. Finally, it retrieves the names of these people. This step-by-step traversal highlights Gremlin’s imperative nature and its ability to express complex graph queries succinctly.

15 / 15: Query Languages: SPARQL

Query Languages: SPARQL

- **SQL-Like Syntax**
 - Familiar structure: `SELECT`, `WHERE`, `FILTER`, etc.
- **Built for RDF Data**
 - Queries triples: subject–predicate–object
- **W3C Standard**
 - Core to Semantic Web and Linked Data applications
- Example:
 - *Find the names of people who know someone named “Alice”*
PREFIX foaf: <...>

SELECT ?personName
WHERE {
 ?person foaf:knows ?friend .
 ?friend foaf:name "Alice" .
 ?person foaf:name ?personName .
}

15 / 15

- **Query Languages: SPARQL**

- **SQL-Like Syntax**

- SPARQL is designed to feel familiar to those who have used SQL. It uses similar keywords like `SELECT`, `WHERE`, and `FILTER`, which makes it easier for people with SQL experience to learn and use SPARQL. This similarity helps in writing queries to retrieve specific data from databases.

- **Built for RDF Data**

- SPARQL is specifically created to query RDF (Resource Description Framework) data. RDF data is structured in triples, which consist of a subject, predicate, and object. This structure is essential for representing information on the Semantic Web, where data is interconnected.

- **W3C Standard**

- SPARQL is a standard developed by the World Wide Web Consortium (W3C). It plays a crucial role in the Semantic Web and Linked Data applications, allowing for the integration and querying of data across different sources on the web.

- **Example**

- *Find the names of people who know someone named “Alice”*

- This example demonstrates a SPARQL query. It uses a **PREFIX** to define a namespace, which helps in shortening URIs. The **SELECT** statement specifies the variable **?personName** to retrieve. The **WHERE** clause contains the pattern to match: it looks for a **?person** who knows a **?friend** with the name “Alice” and retrieves the name of the **?person**. This query highlights how SPARQL can be used to navigate relationships in RDF data.